TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING, PULCHOWK CAMPUS

A Report on

Lab 6

Submitted by:

Sahas Khadka (080BCT071)

Submitted to::

Department of Electronics and Computer Engineering

Pulchowk Campus, Lalitpur

Simple CRUD Application

SmartShop (Django Rest Framework + React JS)

A functional Full-Stack CRUD (Create, Read, Update, Delete) application .

Project Overview

```
lab6/
├── backend/                    # Django REST API
│   ├── products/               # Product Management App
│   │   ├── models.py           # Product Schema
│   │   ├── serializers.py      # DRF Serializers
│   │   ├── views.py            # ModelViewSet (CRUD Logic)
│   │   └── urls.py             # Product API Routes
│   ├── smartshop/              # Project Core Settings
│   │   ├── settings.py         # Django Project Settings
│   │   └── urls.py             # Root URL Configuration
│
├── frontend/                   # React Application
│   ├── src/
│   │   ├── api/                # Fetch API services
│   │   ├── components/         # Reusable UI (ProductCard, etc.)
│   │   ├── pages/              # Main Screens (Home, Form)
│   │   ├── context/            # State Management (Cart)
│   │   ├── App.jsx             # Routing & Layout
│   │   └── main.jsx            # Entry point
│   └── vite.config.js
```

Step 1: Backend Implementation (Django REST Framework)

The backend provides the API infrastructure to handle data persistence.

1. **Model Definition**: Created a Product model in products/models.py with fields for name, description, price , stock, and image_url.

2. **Serialization**: Used serializers.ModelSerializer to automatically convert Django model instances into JSON format.
3. **API Views**: Implemented viewsets.ModelViewSet, which provides the following logic out-of-the-box:

   GET /api/products/ (List all products)

   POST /api/products/ (Create a new product)

   GET /api/products/{id}/ (Retrieve a single product)

   PUT /api/products/{id}/ (Update a product)

   DELETE /api/products/{id}/ (Remove a product)

Step 2: Frontend Implementation (React JS)

The frontend consumes the API and provides an interactive user interface.

API Service Layer: Created src/api/products.js using the Native Fetch API to centralize all network requests to the Django backend.

State Management:

- **Local State**: Used useState and useEffect hooks in components to fetch and store product data.
- **Global State**: Implemented React Context API (CartContext) to manage a persistent shopping cart across different views.

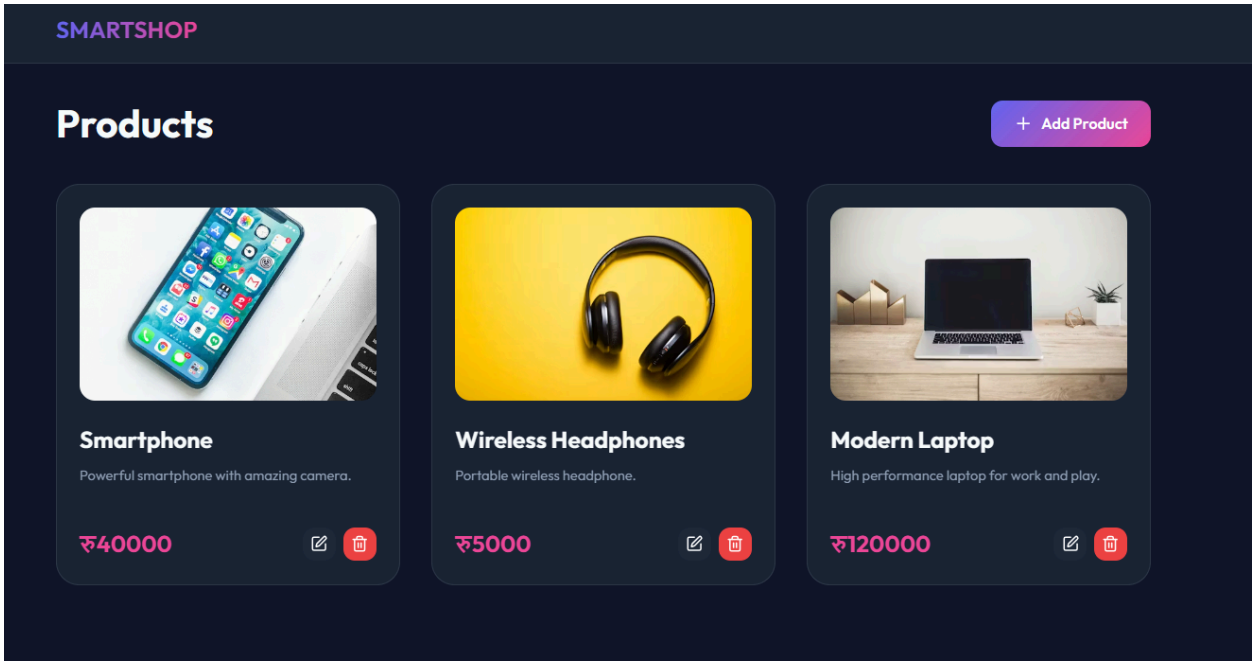
Routing: Utilized react-router to handle navigation between the **Product List (Home)** and the **Product Form (Add/Edit)** pages.

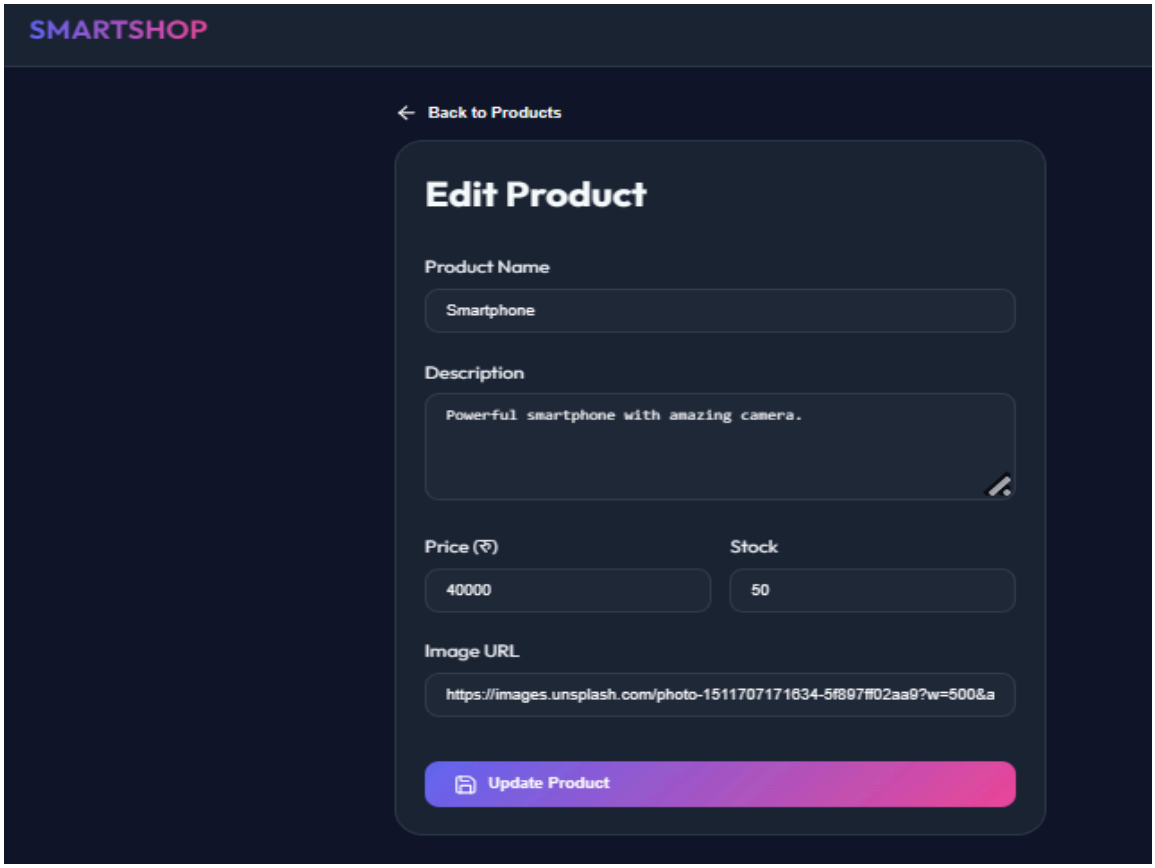
UI Components:

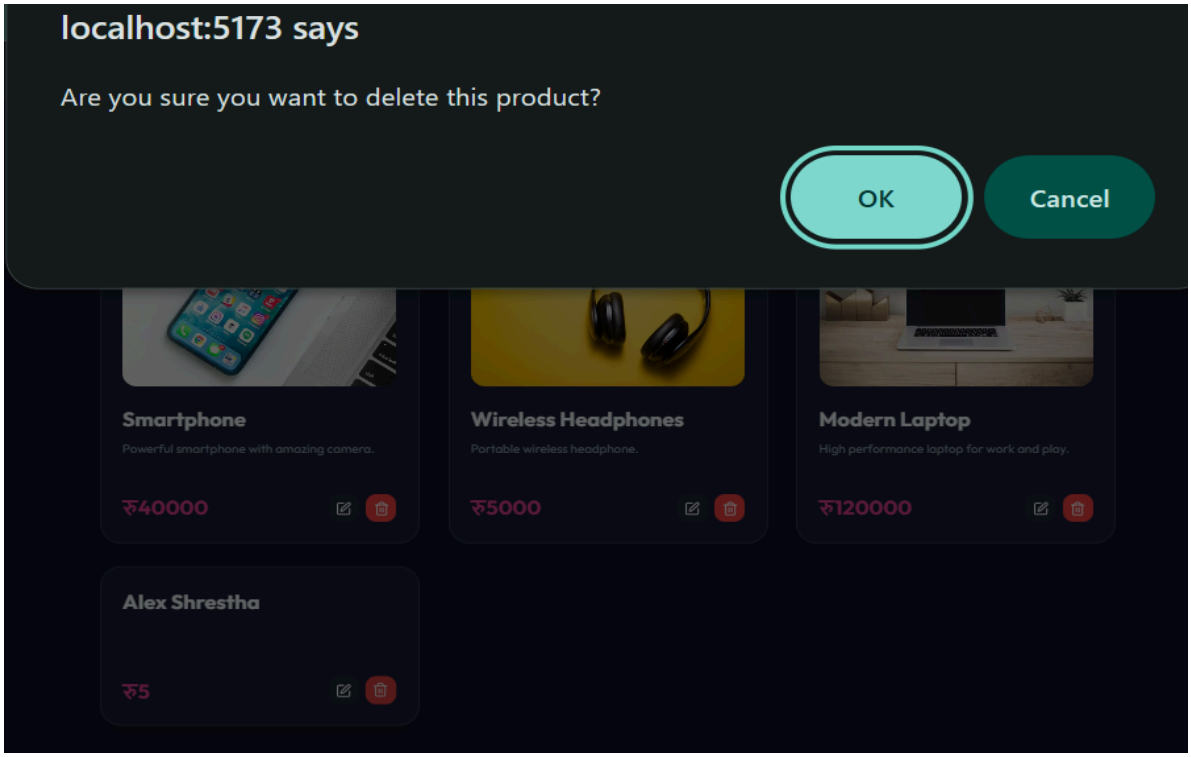
HomePage: Handles **Read** (listing products) and **Delete** actions.

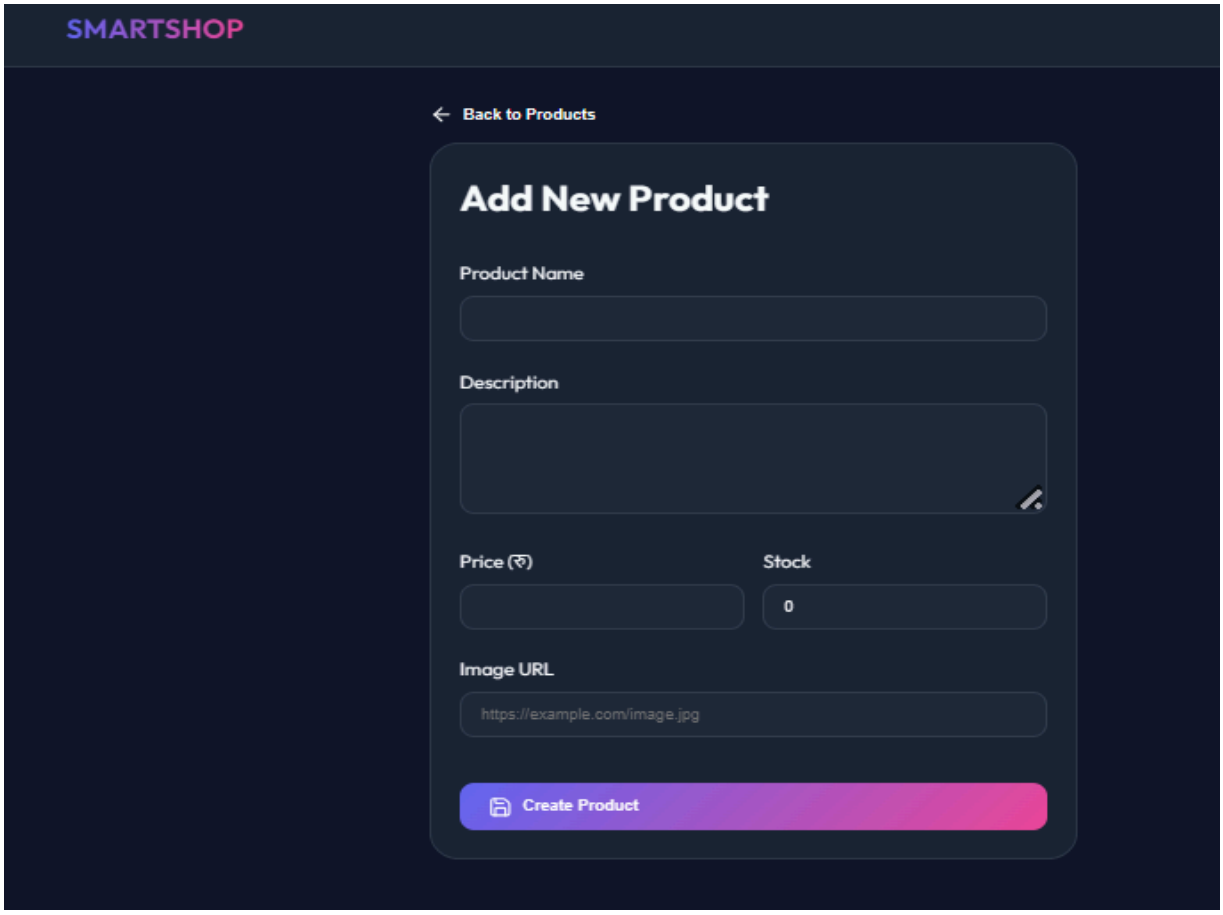
ProductFormPage: Handles both **Create** and **Update** actions using a dynamic form

Output:




SMARTSHOP
Back to Products
Edit Product
Product Name
Smartphone
Description
Powerful smartphone with amazing camera.
Price (रु)
40000
Stock
50
Image URL
https://images.unsplash.com/photo-1511707171634-5f897ff02aa9?w=500&a
Update Product


localhost:5173 says
Are you sure you want to delete this product?
OK
Cancel
Smartphone
Powerful smartphone with amazing camera.
रु40000
Wireless Headphones
Portable wireless headphone.
रु5000
Modern Laptop
High performance laptop for work and play.
रु120000
Alex Shrestha
रु5


SMARTSHOP
Back to Products
Add New Product
Product Name
Description
Price (₹)
Stock
0
Image URL
https://example.com/image.jpg
Create Product